

Making the model answer in a known shape

Week 4, Lecture 1 · Browser Use: How LLM Agents Drive the Web

Summer 2026 · From task to browser action

What we'll cover

- The problem with free-text LLM replies in an agent loop
- Pydantic schemas: turning a Python class into a parse-safe contract
- Function calling: how providers enforce a schema at the API level
- Why structure makes the loop reliable and every failure auditable
- How AgentOutput is built dynamically from the tools registry

Free text vs structured output

Why free text breaks the loop

An LLM asked to control a browser could reply:

"I think I should click the login button on the page."

The framework then has to:

- Parse the prose and guess the intent
- Figure out which element "the login button" refers to
- Handle the fact that every model phrases the same intent differently

None of that is reliable at scale.

Structured output replaces prose with a typed object the framework reads directly.

What changes with a schema

Approach	How the reply arrives	What breaks if the model is wrong
Free text	A string the framework tries to parse	Silent misparsing, wrong action
Structured output	A validated Python object	Validation error, caught immediately

With a schema, the parser *is* the schema. There is nothing to interpret.

Pydantic as the contract

Pydantic v2: fields, types, validators

```
from pydantic import BaseModel, ConfigDict, Field

class AgentOutput(BaseModel):
    model_config = ConfigDict(arbitrary_types_allowed=True, extra="forbid")

    thinking: str | None = None
    evaluation_previous_goal: str | None = None
    memory: str | None = None
    next_goal: str | None = None
    action: list[ActionModel] = Field(..., json_schema_extra={"min_items": 1})
```

`extra="forbid"`: any field the model sends that is not declared here causes an immediate `ValidationError`.

`action` requires at least one entry: an empty action list fails before it reaches the browser.

(browser-use contributors, 2026, agent/views.py)

The contract is hard

`ConfigDict(extra="forbid")` makes the schema a hard boundary:

- Model produces a valid reply → execution proceeds
- Model adds an undeclared field → `ValidationError`, caught immediately
- Model returns empty `action` → validation fails before browser is touched

Compare with `extra="ignore"` (the default): unexpected fields silently vanish. You never know the model tried to send something outside the schema.

For an agent controlling a live browser, "silently ignore" is the wrong policy.

Function calling

How providers enforce the schema

Function calling (also called tool calling or structured outputs) is an API-level parameter:

1. You send the JSON schema of your Pydantic model alongside the messages
2. The provider constrains the model's next reply to match that schema
3. The reply arrives pre-structured, not as a text block

The model does not write text. It fills a form.

Consequence: you do not need a custom parser. You pass the raw reply to

`AgentOutput(**reply)` and Pydantic validates it.

The schema is built dynamically

At agent startup:

```
# From browser_use/agent/service.py
self.ActionModel = self.tools.registry.create_action_model()
self.AgentOutput = AgentOutput.type_with_custom_actions(self.ActionModel)
```

The JSON schema sent to the provider lists **only** the actions in the current tools registry. The model cannot hallucinate an action name that isn't there: the schema doesn't include it.

Add a custom tool → the schema gains a new action type automatically.

Remove a built-in → the schema shrinks automatically.

"Structure is what makes the loop reliable: the model must answer in a known shape, so the framework can always parse and execute it."

Why structure matters for reliability

Every failure is at the schema boundary

Without structure: a bad model reply causes a silent wrong action. You find out when the browser does the wrong thing, possibly steps later.

With structure: a bad model reply causes a `ValidationError` at parse time. The agent can retry the step, log the failure, or stop. The failure is always at the same place.

This means:

- You can add retry logic at one point in the code
- You can log all validation errors in one place
- You can read the raw model reply to diagnose what went wrong

Structure enables inspection

Because every decision is in a known object, you can read any step and reconstruct the agent's state:

```
for item in agent.history.history:  
    output = item.model_output  
    if output:  
        print(output.evaluation_previous_goal)  
        print(output.memory)  
        print(output.next_goal)  
        print([a.model_dump() for a in output.action])
```

No guessing. No parsing. Every field is where it always is.

Take-aways

1. Free-text LLM replies in an agent loop require custom parsing and fail silently. Structured output shifts the failure to the schema boundary.
2. Pydantic v2 with `extra="forbid"` makes the schema a hard contract: invalid replies fail immediately, never silently.
3. Function calling is the API-level mechanism that forces the model to produce a reply that matches the schema.
4. `AgentOutput` is built dynamically from the tools registry each run, so the schema and the available actions are always in sync.
5. Structure makes every decision inspectable: you can read any step's `AgentOutput` and know the agent's full cognitive state at that moment.

What's next

- **Lecture 2 (this week):** Inside AgentOutput: every field, the MessageManager, and the native LLM layer
- **Reading:** Week 4 reading at </c/browser-use-26su/readings/wk04>
- **Section this week:** Capture and dissect real AgentOutput objects from a live multi-step run